

Ex.No: 9

Exercises On Routing

Date:

Aim:

To implement React Programs on Routing

1. Real-Time Job Board Dashboard with Applications Tracking

Create a Real-Time Job Board Dashboard for an online job platform. The system should:

- Track Job Postings: Display new job postings as they are added in real-time.
- Monitor Applications: Track the number of applications per job posting and update in real-time.
- Show Stats: Display the total number of job posts, applications, and successful placements.
- Visualize Data: Use bar or line charts to show the number of applications per position over time.
- Alert System: Notify job seekers and recruiters of new job openings or application status changes.

Code:

[server.js](#):

```
import express from "express";
import http from "http";
import { Server } from "socket.io";
import cors from "cors";

const app = express();
app.use(cors());
app.use(express.json());

const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: "*" }
});

// Fake in-memory DB
```

```

let jobs = [];
let applications = [];

// API to get stats
app.get("/stats", (req, res) => {
  const stats = {
    totalJobs: jobs.length,
    totalApplications: applications.length,
    totalPlacements: applications.filter(a => a.status === "accepted").length
  };
  res.json(stats);
});

// API to post a job
app.post("/jobs", (req, res) => {
  const job = { id: Date.now(), ...req.body, applications: [] };
  jobs.push(job);
  io.emit("job_posted", job);
  res.json(job);
});

// API to apply for a job
app.post("/applications", (req, res) => {
  const appData = { id: Date.now(), ...req.body, status: "applied" };
  applications.push(appData);
  const job = jobs.find(j => j.id === appData.jobId);
  if (job) job.applications.push(appData.id);
  io.emit("application_added", appData);
  res.json(appData);
});

// API to update application status
app.patch("/applications/:id", (req, res) => {
  const appId = parseInt(req.params.id);

```

```

const { status } = req.body;
const appIndex = applications.findIndex(a => a.id === appId);
if (appIndex !== -1) {
  applications[appIndex].status = status;
  io.emit("status_changed", applications[appIndex]);
  res.json(applications[appIndex]);
} else {
  res.status(404).json({ message: "Application not found" });
}
});
server.listen(4000, () => console.log("🚀 Backend running on http://localhost:4000"));
App.jsx:
import { useState, useEffect } from "react";
import { useDispatch, useSelector } from "react-redux";
import io from "socket.io-client";
import axios from "axios";
import { addJob, updateJobApplications } from "./jobSlice";
import { addApplication } from "./applicationSlice";
import { ToastContainer, toast } from "react-toastify";
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip } from "recharts";
import "react-toastify/dist/ReactToastify.css";

// connect socket
const socket = io("http://localhost:4000", {
  transports: ["websocket"], // force websocket
});

export default function App() {
  const dispatch = useDispatch();
  const jobs = useSelector((state) => state.jobs);
  const applications = useSelector((state) => state.applications);

```

```
const [stats, setStats] = useState({
  totalJobs: 0,
  totalApplications: 0,
  totalPlacements: 0,
});
```

```
const [jobTitle, setJobTitle] = useState("");
```

```
useEffect(() => {
  socket.on("connect", () => {
    console.log(" Connected to socket:", socket.id);
  });
});
```

```
socket.on("job_posted", (job) => { console.log("
  Job received:", job); dispatch(addJob(job));
  toast.info(`New Job Posted: ${job.title}`);
  setStats((prev) => ({ ...prev, totalJobs: prev.totalJobs + 1 }));
});
```

```
socket.on("application_added", (application) => { console.log("
  Application received:", application);
  dispatch(updateJobApplications({ jobId: application.jobId, application }));
  dispatch(addApplication(application));
  toast.success(`New Application for Job ID: ${application.jobId}`);
  setStats((prev) => ({
    ...prev,
    totalApplications: prev.totalApplications + 1,
  }));
});
```

```
socket.on("status_changed", (application) => {
  console.log(" Status updated:", application);
});
```

```

toast.info(
  `Application ${application.id} status changed to ${application.status}`
);
if (application.status === "accepted") {
  setStats((prev) => ({
    ...prev,
    totalPlacements: prev.totalPlacements + 1,
  }));
}
});

return () => {
  socket.off("connect");
  socket.off("job_posted");
  socket.off("application_added");
  socket.off("status_changed");
};
}, [dispatch]);

// Fetch initial stats on mount
useEffect(() => {
  const fetchStats = async () => {
    try {
      const res = await axios.get("http://localhost:4000/stats");
      setStats(res.data);
    } catch (err) {
      console.error("Error fetching stats:", err);
    }
  };
  fetchStats();
}, []);

// Create Job API call
const createJob = async () => {

```

```

if (!jobTitle) return;
try {
  const res = await axios.post("http://localhost:4000/jobs", {
    title: jobTitle,
    company: "MyCompany",
  });
  setJobTitle("");
  console.log("Job posted:", res.data);
} catch (err) {
  console.error("Error posting job:", err);
}
};

// Apply for a job
const applyForJob = async (jobId) => {
  try {
    const res = await axios.post("http://localhost:4000/applications", {
      jobId,
      userId: Date.now(),
    });
    console.log("Applied:", res.data);
  } catch (err) {
    console.error("Error applying:", err);
  }
};

// Chart data
const chartData = jobs.map((job) => ({
  name: job.title,
  applications: job.applications?.length || 0,
})));

return (
  <div style={{ padding: 20 }}>

```

```
<h1> Real-Time Job Board Dashboard</h1>
<h2> Stats</h2>
<p>Total Jobs: {stats.totalJobs}</p>
<p>Total Applications: {stats.totalApplications}</p>
<p>Total Placements: {stats.totalPlacements}</p>
```

```
{/* Job Form */}
<h2>Post a Job</h2>
<input
  type="text"
  placeholder="Job title"
  value={jobTitle}
  onChange={(e) => setJobTitle(e.target.value)}
/>
<button onClick={createJob}>Add Job</button>
```

```
{/* Jobs List */}
<h2>Jobs</h2>
<ul>
  {jobs.map((job) => (
    <li key={job.id}>
      {job.title} — Applications: {job.applications?.length || 0}
      <button onClick={() => applyForJob(job.id)}>Apply</button>
    </li>
  ))}
</ul>
```

```
{/* Debug */}
<h2>Jobs (Debug JSON)</h2>
<pre>{JSON.stringify(jobs, null, 2)}</pre>
```

```
{/* Chart */}
<h2>Applications Chart</h2>
```

```

<BarChart width={500} height={300} data={chartData}>
  <CartesianGrid strokeDasharray="3 3" />
  <XAxis dataKey="name" />
  <YAxis />
  <Tooltip />
  <Bar dataKey="applications" fill="#8884d8" />
</BarChart>

```

```

<ToastContainer />
</div>

```

```
);
```

```
}
```

```
jobSlice.js:
```

```
import { createSlice } from "@reduxjs/toolkit";
```

```

const jobSlice = createSlice({
  name: "jobs",
  initialState: [],
  reducers: {
    addJob: (state, action) => {
      state.push(action.payload);
    },
    updateJobApplications: (state, action) => {
      const { jobId, application } = action.payload;
      const job = state.find(j => j.id === jobId);
      if (job) {
        if (!job.applications) job.applications = [];
        job.applications.push(application);
      }
    }
  }
});

```

```

export const { addJob, updateJobApplications } = jobSlice.actions;
export default jobSlice.reducer;

```


[applicationSlice.js](#):

```
import { createSlice } from "@reduxjs/toolkit";

const applicationSlice = createSlice({
  name: "applications",
  initialState: [],
  reducers: {
    addApplication: (state, action) => {
      state.push(action.payload);
    }
  }
});

export const { addApplication } = applicationSlice.actions;
export default applicationSlice.reducer;
```

[store.js](#):

```
import { configureStore } from "@reduxjs/toolkit";
import jobReducer from "../jobSlice";
import applicationReducer from "../applicationSlice";

const store = configureStore({
  reducer: {
    jobs: jobReducer,
    applications: applicationReducer
  }
});

export default store;
```

Output:

Real-Time Job Board Dashboard

Stats

Total Jobs: 4
Total Applications: 3
Total Placements: 0

Post a Job

Job title

Add Job

Jobs

- data analyst — Applications: 2

Apply
- we developer — Applications: 1

Apply
- devops engineer — Applications: 0

Apply

Jobs (Debug JSON)

```
[
  {
    "id": 1757349391308,
    "title": "data analyst",
    "company": "MyCompany",
    "applications": [
      {
        "id": 1757349427361,
        "jobId": 1757349391308,
        "userId": 1757349427348,
        "status": "applied"
      }
    ]
  }
]
```

New Job Posted: devops engineer

Real-Time Job Board Dashboard

Stats

Total Jobs: 4
Total Applications: 4
Total Placements: 0

Post a Job

Job title

Add Job

Jobs

- data analyst — Applications: 2

Apply
- we developer — Applications: 1

Apply
- devops engineer — Applications: 1

Apply

Jobs (Debug JSON)

```
[
  {
    "id": 1757349391308,
    "title": "data analyst",
    "company": "MyCompany",
    "applications": [
      {
        "id": 1757349427361,
        "jobId": 1757349391308,
        "userId": 1757349427348,
        "status": "applied"
      }
    ],
  }
]
```

✓

New Application for Job ID:
1757349485624

Jobs (Debug JSON)

```
[
  {
    "id": 1757349391308,
    "title": "data analyst",
    "company": "MyCompany",
    "applications": [
      {
        "id": 1757349427361,
        "jobId": 1757349391308,
        "userId": 1757349427340,
        "status": "applied"
      },
      {
        "id": 1757349460304,
        "jobId": 1757349391308,
        "userId": 1757349460292,
        "status": "applied"
      }
    ]
  },
  {
    "id": 1757349417867,
    "title": "we developer",
    "company": "MyCompany",
    "applications": [
      {
        "id": 1757349431295,
        "jobId": 1757349417867,
        "userId": 1757349431290,
        "status": "applied"
      }
    ]
  },
  {
    "id": 1757349485624,
    "title": "devops engineer",
    "company": "MyCompany",
    "applications": [
      {
        "id": 1757349491904,
        "jobId": 1757349485624,
        "userId": 1757349491895,
        "status": "applied"
      }
    ]
  }
]
```

```
{
  "id": 1757349431295,
  "jobId": 1757349417867,
  "userId": 1757349431290,
  "status": "applied"
},
{
  "id": 1757349485624,
  "title": "devops engineer",
  "company": "MyCompany",
  "applications": [
    {
      "id": 1757349491904,
      "jobId": 1757349485624,
      "userId": 1757349491895,
      "status": "applied"
    }
  ]
}
]
```

Applications Chart



2. Real-Time Restaurant Reservation and Table Availability Dashboard

Develop a Real-Time Restaurant Reservation and Table Availability Dashboard for managing restaurant reservations. The system should:

- Track Reservations: Monitor incoming reservation requests in real time.
- Display Table Availability: Show the real-time status of available, booked, and pending tables.
- Monitor Waitlist: Track the number of people waiting for a table and their estimated wait times.
- Real-Time Stats: Display the total number of reservations, cancellations, and available seats at any given time.
- Alert System: Send notifications to staff when a table becomes available or a reservation is approaching.

Code:

[server.js](#):

```
import express from "express";
import http from "http";
import { Server } from "socket.io";
import cors from "cors";

const app = express();
app.use(cors());
app.use(express.json());

const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*" } });

// In-memory DB
let tables = [
  { id: 1, seats: 2, status: "available" },
  { id: 2, seats: 4, status: "available" },
  { id: 3, seats: 4, status: "available" }
];

let reservations = [];
let waitlist = [];
```

```

function emitUpdate() {
  io.emit("update", { tables, reservations, waitlist });
}

// API - Make a reservation
app.post("/reserve", (req, res) => {
  const { name, size } = req.body;
  const id = Date.now();
  const table = tables.find(t => t.status === "available" && t.seats >= size);

  if (table) {
    table.status = "booked";
    reservations.push({ id, name, size, tableId: table.id, status: "confirmed" });
  } else {
    waitlist.push({ id, name, size });
    reservations.push({ id, name, size, tableId: null, status: "waitlisted" });
  }

  emitUpdate();
  res.json({ success: true });
});

// API - Cancel reservation
app.post("/cancel/:id", (req, res) => {
  const id = parseInt(req.params.id);
  reservations = reservations.map(r =>
    r.id === id ? { ...r, status: "cancelled" } : r
  );
  tables.forEach(t => {
    if (t.id === reservations.find(r => r.id === id)?.tableId) {
      t.status = "available";
    }
  });
});

```

```

    waitlist = waitlist.filter(w => w.id !== id);
    emitUpdate();
    res.json({ success: true });
  });

  io.on("connection", (socket) => {
    console.log("Client connected");
    socket.emit("update", { tables, reservations, waitlist });
  });

  server.listen(4001, () => console.log("Backend running on http://localhost:4001"));

```

App.jsx

```

import React, { useEffect, useState } from "react";
import io from "socket.io-client";
import axios from "axios";
const socket = io("http://localhost:4001");
export default function App() {
  const [tables, setTables] = useState([]);
  const [reservations, setReservations] = useState([]);
  const [waitlist, setWaitlist] = useState([]);

  const [name, setName] = useState("");
  const [size, setSize] = useState(2);

  useEffect(() => {
    socket.on("update", (data) => {
      setTables(data.tables);
      setReservations(data.reservations);
      setWaitlist(data.waitlist);
    });
    return () => socket.off("update");
  });

```

```
}, []);
```

```
const reserve = async () => {  
  await axios.post("http://localhost:4001/reserve", { name, size });  
  setName("");  
  setSize(2);  
};
```

```
const cancel = async (id) => {  
  await axios.post(`http://localhost:4001/cancel/${id}`);  
};
```

```
return (  
  <div style={{ padding: 20, fontFamily: "sans-serif" }}>  
    <h1>🍽️ Restaurant Dashboard</h1>  
  
    <h2>Make a Reservation</h2>  
    <input  
      placeholder="Name"  
      value={name}  
      onChange={(e) => setName(e.target.value)}  
    />  
    <input  
      type="number"  
      value={size}  
      onChange={(e) => setSize(parseInt(e.target.value))}  
      style={{ marginLeft: 8, width: 50 }}  
    />  
    <button onClick={reserve} style={{ marginLeft: 8 }}>Reserve</button>  
  
    <h2>Tables</h2>  
    <ul>  
      {tables.map(t => (  
        <li key={t.id}>
```

```

        Table {t.id} ({t.seats} seats): {t.status}
      </li>
    )}
  </ul>

  <h2>Reservations</h2>
  <ul>
    {reservations.map(r => (
      <li key={r.id}>
        {r.name} — {r.size} — {r.status}
        {r.status !== "cancelled" && (
          <button onClick={() => cancel(r.id)} style={{ marginLeft: 8 }}>
            Cancel
          </button>
        )}
      </li>
    )}
  </ul>

  <h2>Waitlist ({waitlist.length})</h2>
  <ul>
    {waitlist.map(w => (
      <li key={w.id}>{w.name} — {w.size}</li>
    )}
  </ul>
</div>

);
}

```

Output:

Restaurant Dashboard

Make a Reservation

Name	2	Reserve
-----------------------------------	--------------------------------	--

Tables

- Table 1 (2 seats): booked
- Table 2 (4 seats): booked
- Table 3 (4 seats): booked

1. Real-Time Customer Support Chat Dashboard

Design a Real-Time Customer Support Chat Dashboard for handling live customer support queries. The system should:

- Track Support Requests: Monitor incoming chat requests from customers in real time.
- Assign Support Agents: Automatically assign support tickets to available agents or show the number of pending requests.
- Real-Time Stats: Display metrics like response time, number of tickets resolved, and average support duration.

- Visualize Data: Use charts to show the number of open and closed tickets, average wait times, and agent performance.
- Alert System: Notify agents when a new high-priority support request is received or when response times exceed predefined limits.

Code:

[server.js](#)

```
import express from "express";

import http from "http";

import { Server } from "socket.io";

import cors from "cors";

const app = express();

app.use(cors());

app.use(express.json());

const server = http.createServer(app);

const io = new Server(server, { cors: { origin: "*" } });

// In-memory storage

let tickets = []; // { id, customerName, message, status: 'open'|'closed', agent: string|null, timestamp }

let agents = ["Alice", "Bob"]; // Example agents

function emitUpdate() {

  io.emit("update", tickets);

}
```

```
// API to create a new ticket

app.post("/ticket", (req, res) => {

  const { customerName, message } = req.body;

  const id = Date.now();

  const availableAgent = agents.find(a => !tickets.some(t => t.agent === a && t.status === "open"));

  const ticket = {

    id,

    customerName

    , message,

    status: "open",

    agent: availableAgent || null,

    timestamp: new Date().toISOString()

  };

  tickets.push(ticket);

  io.emit("new_ticket", ticket);

  emitUpdate();

  res.json(ticket);

});
```

```
// API to close a ticket

app.post("/ticket/:id/close", (req, res) => {

  const id = parseInt(req.params.id);

  const t = tickets.find(ticket => ticket.id === id);

  if (!t) return res.status(404).json({ message: "Ticket not found" });

  t.status = "closed";
```

```
io.emit("ticket_closed", t);  
emitUpdate();  
res.json(t);  
});
```

```
// Socket.io connection
```

```
io.on("connection", (socket) => {  
  console.log("Client connected:", socket.id);  
  socket.emit("update", tickets);  
});
```

```
server.listen(4002, () => console.log("Backend running on http://localhost:4002"));
```

```
App.jsx
```

```
import React, { useEffect, useState } from "react";  
import io from "socket.io-client";  
import axios from "axios";
```

```
const socket = io("http://localhost:4002");
```

```
export default function App() {  
  const [tickets, setTickets] = useState([]);  
  const [customerName, setCustomerName] = useState("");  
  const [message, setMessage] = useState("");
```

```

useEffect(() => {
  socket.on("update", setTickets);
  socket.on("new_ticket", (t) => alert(`New ticket from ${t.customerName}`));
  socket.on("ticket_closed", (t) => alert(`Ticket closed: ${t.customerName}`));

  return () => socket.off();
}, []);

```

```

const createTicket = async () => {
  if (!customerName || !message) return;
  await axios.post("http://localhost:4002/ticket", { customerName, message });
  setCustomerName("");
  setMessage("");
};

```

```

const closeTicket = async (id) => {
  await axios.post(`http://localhost:4002/ticket/${id}/close`);
};

```

// Simple stats

```

const total = tickets.length;
const open = tickets.filter(t => t.status === "open").length;
const closed = tickets.filter(t => t.status === "closed").length;

```

```

return (

```

```
<div style={{ padding: 20, fontFamily: "sans-serif" }}>
```

```
<h1><img alt="speech bubble icon" data-bbox="188 121 218 141"/> Customer Support Dashboard</h1>
```

```
<h2>Create Ticket</h2>
```

```
<input placeholder="Customer Name" value={customerName} onChange={e =>
setCustomerName(e.target.value)} />
```

```
<input placeholder="Message" value={message} onChange={e =>
setMessage(e.target.value)} style={{ marginLeft: 8 }} />
```

```
<button onClick={createTicket} style={{ marginLeft: 8 }}>Send</button>
```

```
<h2>Stats</h2>
```

```
<p>Total Tickets: {total}</p>
```

```
<p>Open Tickets: {open}</p>
```

```
<p>Closed Tickets: {closed}</p>
```

```
<h2>Tickets</h2>
```

```
<ul>
```

```
{tickets.map(t => (
```

```
<li key={t.id}>
```

```
{t.customerName} — {t.message} — {t.status} — Agent: {t.agent || "unassigned"}
```

```
{t.status === "open" && (
```

```
<button onClick={() => closeTicket(t.id)} style={{ marginLeft: 8 }}>Close</button>
```

```
)}
```

```
</li>
```

```
))}
```

```
</ul>
```

```
</div>

);

}
```

Output:

localhost:5173 says
New ticket from Alice

OK



Customer Support Dashboard

Create Ticket

Stats

Total Tickets: 0

Open Tickets: 0

Closed Tickets: 0

Tickets



Customer Support Dashboard

Create Ticket

Stats

Total Tickets: 1

Open Tickets: 1

Closed Tickets: 0

Tickets

- Alice — I need help — open — Agent: unassigned

Problem Analysis (10)	Coding & Implementation (10)	Time Management (10)	Viva (10)	Total (40)

Result:

Thus the program on routers has been executed successfully and the output has been verified.